Computer understand machine language, i.e very difficult to understand and difficult to programming. Therefore, require a bridge gap.
As we already know programming language is different from machine language, that computer can understand.
This is where compiler comes in.
A compiler translates (compile) a program written in a high level language program language (HLL PL) that is suitable for human programmers into low level machine language that is required by computer.
Or
Compiler is a process that will convert source into object code.
During this process, the compiler will attempts to spot and report obvious programmers mistakes.

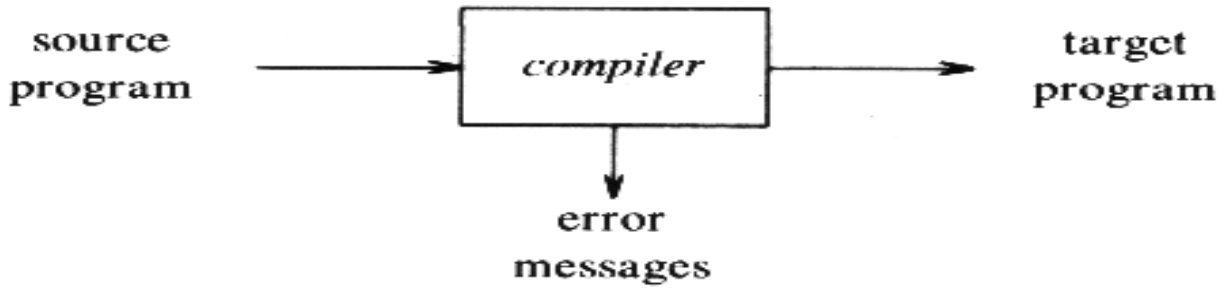


Fig. 1.1. A compiler.

First real compiler - FORTRAN COMPILER of late 1950.

Why High Level Programming Language (HLPL) is more beneficial as compared to Low level Programming Language (LLPL)?
Using high level language for programming has a large impact on, how fast programs can be developed. The main reason for this are:

- Compared to machine language, the notation used by programming language is closer to the way human think about the problem
- The compiler can spot some obvious programming mistakes
- Program written in High Level Programming Language tends to shorter then equivalent program written in machine language
- Another advantage of using HLPL is that the same program can be compiled to many different machine language and hence be brought to run on different machine
- While in LLPL, program execute and written for specific machine

Any compiler must perform two major tasks:

- **Analysis – of** Source Program
- **Synthesis –** of machine language program

Hybrid (JVM)
Source Program → Translator → Intermediate Program Input → Virtual Machine → Output

Need of translators

The translator is used to translate the source code into machine language, which the machine can understand. While translating the source code, the machine detects the errors that are known as syntax errors.

Roles of translator are:

• Translating the high-level language program input into an equivalent machine language program.

• Providing diagnostic messages wherever the programmer violates specification of the high-level language program.

Different type of translators

The different types of translator are as follows:

Compiler

Compiler is a translator which is used to convert programs in high-level language to low-level language. It translates the entire program and also reports the errors in source program encountered during the translation.

Interpreter

Interpreter is a translator which is used to convert programs in high-level language to low-level language. Interpreter translates line by line and reports the error once it encountered during the translation process.

It directly executes the operations specified in the source program when the input is given by the user.

It gives better error diagnostics than a compiler.

Assembler

Assembler is a translator which is used to translate the assembly language code into machine language code.

Phases of Compiler : To defined well-defined interface.

Conceptually, these phases operate in sequence, each phase the output from previous phase as its input.

Each phase handled by separate module. Some of these modules are written by hand while other may be generated from specification after some of modules can be shared between several compilers.

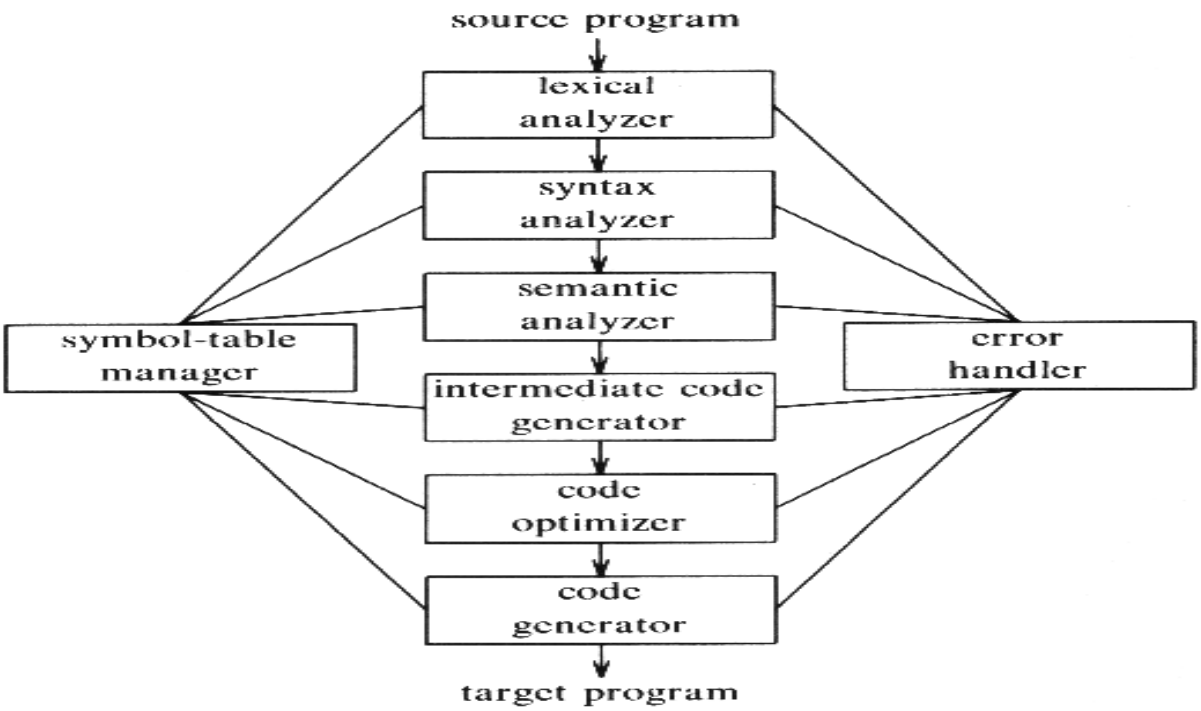


Fig. 1.9. Phases of a compiler.

Lexical Analysis (Scanner)

- Lexical analysis is the first phase of compiler which is also termed as scanning.
- This is initial part of reading and analysis the program text; Text is read and divided into token, each of which corresponds to a symbol in the programming language (PL) e.g. variable name, keywords or numbers.
- It read the source program one character at one time, and those character are grouped into a sequence of automatic unit called token.
- Source program is scanned to read the stream of characters and those characters are grouped to form a sequence called lexemes which produces token as output.
- **Token:** Token is a sequence of characters that represent lexical unit, which matches with the pattern, such as keywords, operators, identifiers etc.
- **Lexeme:** Lexeme is instance of a token i.e., group of characters forming a token. ,
- **Pattern:** Pattern describes the rule that the lexemes of a token takes. It is the structure that must be matched by strings.
- Once a token is generated the corresponding entry is made in the symbol table.

Input: stream of characters
Output: Token
Token Template: <token-name, attribute-value>
(eg.) c=a+b*5;

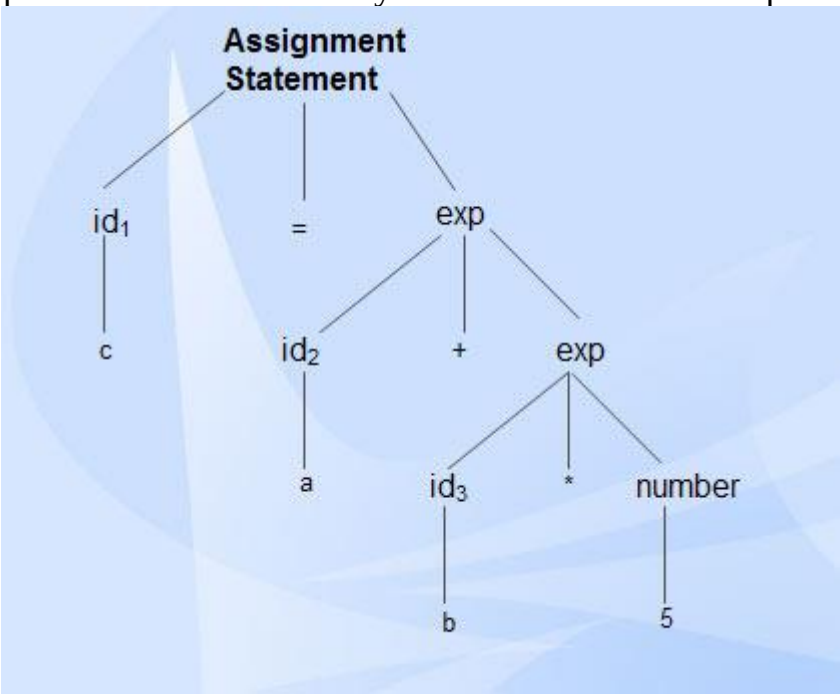
Lexemes and tokens

Lexemes	Tokens
c	Identifier
=	assignment symbol
a	Identifier
+	+ (addition symbol)
b	Identifier
*	* (multiplication symbol)
5	5 (number)

Hence, <id, 1><=>< id, 2>< +><id, 3 >< * >< 5>

Syntax Analysis

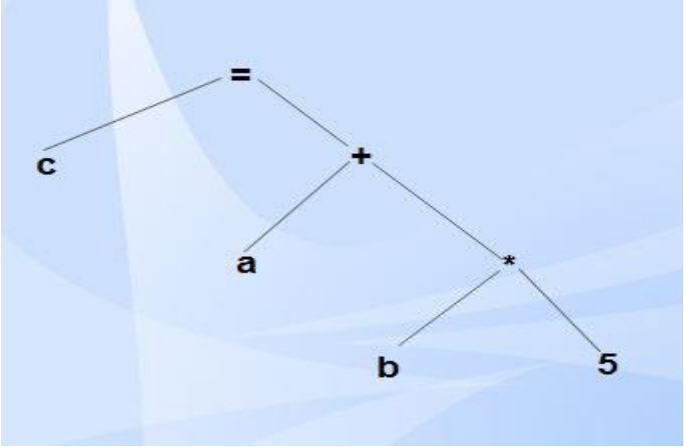
- Syntax analysis is the second phase of compiler which is also called as parsing.
- This phase takes the list of tokens produced by the lexical analysis and arrange these in a tree structure (Syntax Tree) that reflects the structure of program.
- Parser converts the tokens produced by lexical analyzer into a tree like representation called parse tree.
- A parse tree describes the syntactic structure of the input.



• Syntax tree is a compressed representation of the parse tree in which the operators appear as interior nodes and the operands of the operator are the children of the node for that operator.

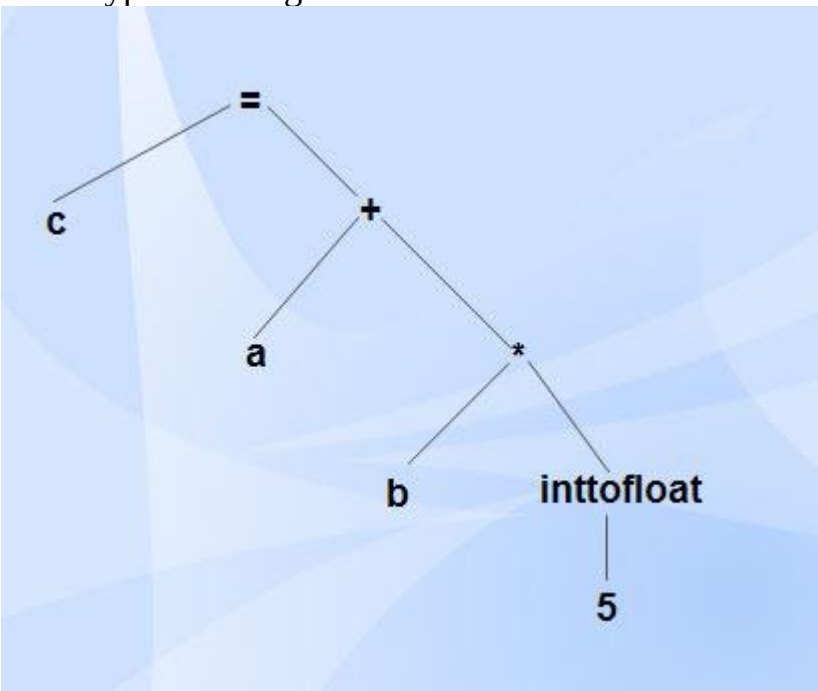
Input: *Tokens*

Output: *Syntax tree*



Semantic Analysis

- Semantic analysis is the third phase of compiler.
- This phase analysis the syntax tree to determine if program violates certain consistency requirements.
- For example: if a variable is used, but not declared or if it is used in the context that does not make sense given the type variable such as trying to use a Boolean value as a function pointer.
- It checks for the semantic consistency.
- Type information is gathered and stored in symbol table or in syntax tree.
- Performs type checking.



Intermediate Code Generation

- This phase bridges the analysis and synthesis phase of translation
- **It produces intermediate representations for the source program which are of the following forms:**
  - **Postfix notation**
  - **Three address code**
  - **Syntax tree**

- Most commonly used form is the three address code.

  t_1 = int to float (5)
  t_2 = id_3* tl
  t_3 = id_2 + t_2
  id_1 = t_3

Properties of intermediate code

• It should be easy to produce.
• It should be easy to translate into target program.

Code Optimization

- This is optional phase described to improve the intermediate code, so that the output runs faster and takes less space.
- Code optimization phase gets the intermediate code as input and produces optimized intermediate code as output.
- It results in faster running machine code.
- It can be done by reducing the number of lines of code for a program.
- This phase reduces the redundant code and attempts to improve the intermediate code so that faster-running machine code will result.
- During the code optimization, the result of the program is not affected.
- **To improve the code generation, the optimization involves**
  - o Deduction and removal of dead code (unreachable code).
  - o Calculation of constants in expressions and terms.
  - o Collapsing (failing) of repeated expression into temporary string.
  - o Loop unrolling.
  - o Moving code outside the loop.
  - o Removal of unwanted temporary variables.

    t_1 = id_3* 5.0
    id_1 = id_2 + t_1

Code Generation

• Code generation is the final phase of a compiler.

• It gets input from code optimization phase and produces the target code or object code as result.
• Intermediate instructions are translated into a sequence of machine instructions that perform the same task.
• The code generation involves
  o Allocation of register and memory.
  o Generation of correct references.
  o Generation of correct data types.
  o Generation of missing code.

LDF R_2, id_3
MULF R_2, # 5.0
LDF R_1, id_2
ADDF R_1, R_2
STF id_1, R_1

Symbol Table Management

- It is also known as book keeping.
- **This is the portion to keep the names used by the program records, essential information, data structure is used to store all information, called symbol table.**
- Symbol table is used to store all the information about identifiers used in the program.
- It is a data structure containing a record for each identifier, with fields for the attributes of the identifier.
- It allows finding the record for each identifier quickly and to store or retrieve data from that record.
- Whenever an identifier is detected in any of the phases, it is stored in the symbol table.

Example
int a, b; float c; char z;

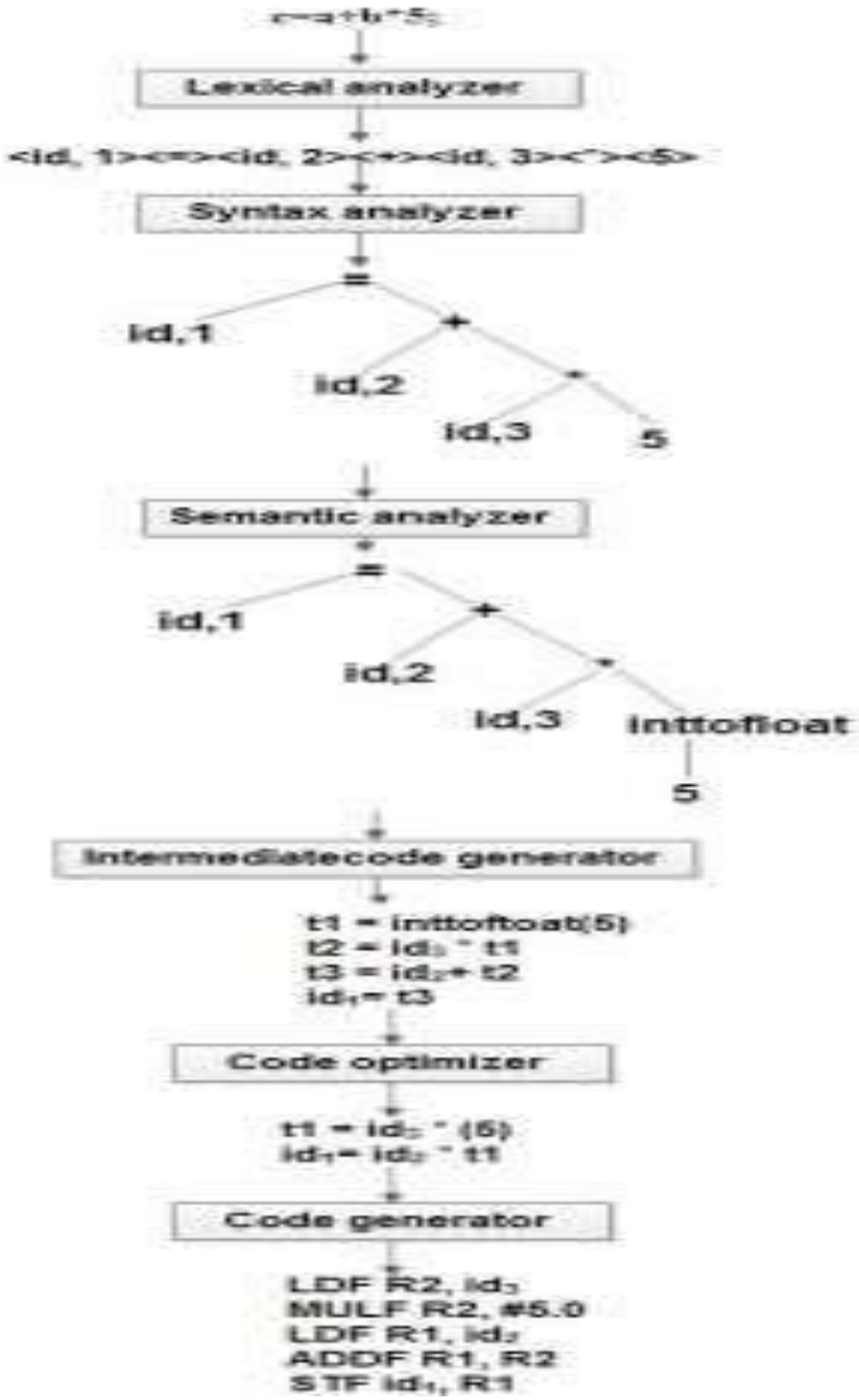
Symbol name	**Type**	**Address**
A	Int	1000
B	Int	1002
C	Float	1004
Z	char	1008

Example
extern double test (double x);
double sample (int count)

```
{
  double sum= 0.0;
    for (int i = 1; i < = count; i++)
      sum+= test((double) i);
      return sum;
  }
```

Symbol name	**Type**	**Scope**
Test	function, double	Extern
X	double	function parameter
Sample	function, double	Global
Count	int	function parameter
Sum	double	block local
I	int	for-loop statement

Prof(Dr) Anil



To record the identifier in source program (identifier is detected by lexical analysis and then is used to stored in the symbol table.)

To collect the attributes of identifiers:

- Storage Allocation: Memory Address
- Types: Scope (where it is valid, local or global)

- Arguments (in case of procedure names): Argument number and types
    - Call by reference and call by address
    - Return type

Error Handling

- Each phase can encounter errors.
- After detecting an error, a phase must handle the error so that compilation can proceed.
- In lexical analysis, errors occur in separation of tokens.
- In syntax analysis, errors occur during construction of syntax tree.
- In semantic analysis, errors may occur at the following cases:
    - When the compiler detects constructs that have right syntactic structure but no meaning
    - During type conversion.
- In code optimization, errors occur when the result is affected by the optimization.
- In code generation, it shows error when code is missing etc.

Figure illustrates the translation of source code through each phase, considering the statement

c =*a*+ *b* * 5.

Error Encountered in Different Phases

Each phase can encounter errors. After detecting an error, a phase must somehow deal with the error, so that compilation can proceed.
A program may have the following kinds of errors at various stages:

1. **Lexical Errors:** It includes incorrect or misspelled name of some identifier i.e., identifiers typed incorrectly. These errors occur due to not reachable code-infinite loop.
2. **Syntactical Errors:** It includes missing semicolon or unbalanced parenthesis. Syntactic errors are handled by syntax analyzer (parser).
   When an error is detected, it must be handled by parser to enable the parsing of the rest of the input. In general, errors may be expected at various stages of compilation but most of the errors are syntactic errors and hence the parser should be able to detect and report those errors in the program.

The goals of error handler in parser are:

- Report the presence of errors clearly and accurately.
- Recover from each error quickly enough to detect subsequent errors.
- Add minimal overhead to the processing of correcting programs.

There are four common error-recovery strategies that can be implemented in the parser to deal with errors in the code.

- Panic mode.
- Statement level.

- Error productions.
- Global correction.

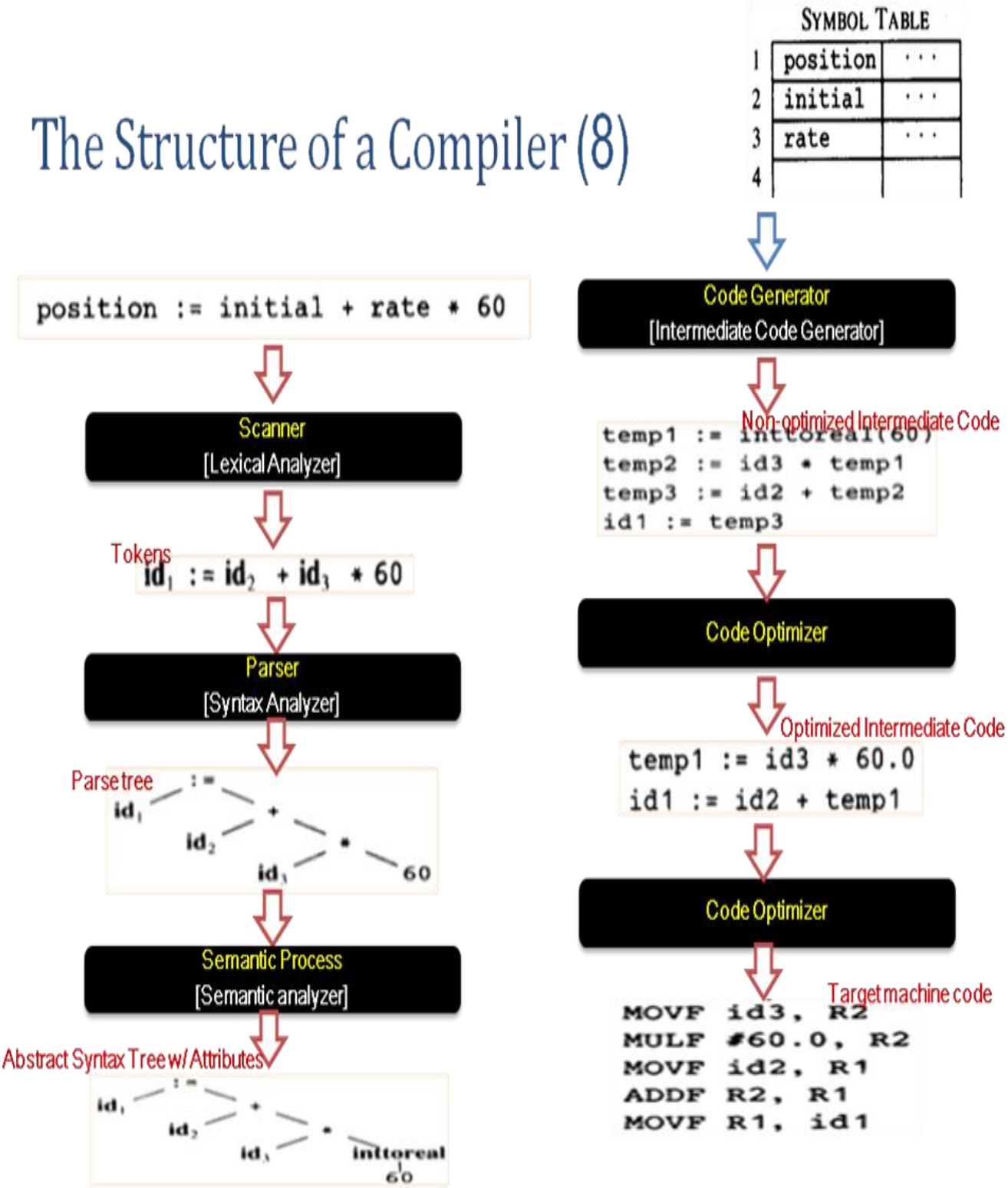
Semantically Errors

These errors are a result of incompatible value assignment. The semantic errors that the semantic analyzer is expected to recognize are:

- Type mismatch.
  - Undeclared variable.
  - Reserved identifier misuse.
  - Multiple declaration of variable in a scope.
  - Accessing an out of scope variable.
  - Actual and formal parameter mismatch.

Prof(Dr) Anil Kumar

The Structure of a Compiler (8)



15

Compiler Construction Tools

Some commonly used compiler-construction tools. include

1. Parser generators.
2. Scanner generators.
3. Syntax-directed translation engines.
4. Automatic code generators.
5. Data-flow analysis engines.
6. Compiler-construction toolkits.

Parser Generators

Input: Grammatical description of a programming language

Output: Syntax analyzers.

Parser generator takes the grammatical description of a programming language and produces a syntax analyzer.

Scanner Generators

Input: Regular expression description of the tokens of a language

Output: Lexical analyzers.

Scanner generator generates lexical analyzers from a regular expression description of the tokens of a language.

Syntax-directed Translation Engines

Input: Parse tree.

Output: Intermediate code.

Syntax-directed translation engines produce collections of routines that walk a parse tree and generates intermediate code.

Automatic Code Generators

Input: Intermediate language.

Output: Machine language.

Code-generator takes a collection of rules that define the translation of each operation of the intermediate language into the machine language for a target machine.

Data-flow Analysis Engines

Data-flow analysis engine gathers the information, that is, the values transmitted from one part of a program to each of the other parts. Data-flow analysis is a key part of code optimization.

Compiler Construction Toolkits

The toolkits provide integrated set of routines for various phases of compiler. Compiler construction toolkits provide an integrated set of routines for construction of phases of compiler.

Lexical analysis:

A lexical analyzer must be able to do three things:

- Remove all whitespace and comments.
- Identify tokens within a string.
- Return the lexeme of a found token, as well as the line number it was found on.

Pattern

- The set of strings described by a rule called a pattern associated with the token."
- The pattern is said to match each string in the set.
- For example, the pattern to describe an identifier (a variable) is a string of letters, numbers, or underscores, beginning with a non-number.
- The pattern for identifier token, ID, is

  ID → letter (letter | digit)*

Patterns are typically described using regular expressions.(RE)

Lexeme

- "A lexeme is a sequence of characters in the source program that is matched by the pattern for a token"

For example, the pattern for the Relation_op token contains six lexemes (=, < >, <, < =, >, >=) so the lexical analyzer should return a relation_op token to parser whenever it sees any one of the six.

For example:

Input: count = 123

Tokens:

identifier : *Rule*: "letter followed by …"

Lexeme: count

assg_op : *Rule*: =

Lexeme: =

integer_const : *Rule*: "digit followed by …"

Lexeme: 123

+Example:

Const pi = 3.1416;

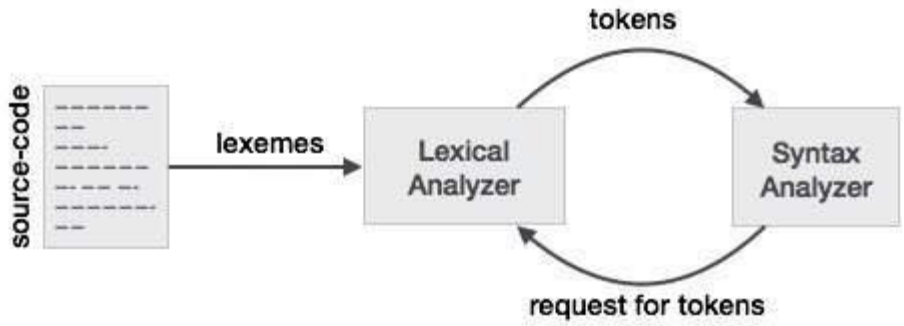
In the example above ,when the character sequence "pi" appears in the source program,a token representing an identifier is retuned to the parser(ID).

The returning of a token is often implemented by passing an integer corresponding to the token.

Function Lexical Analysis

Lexical analysis is the first phase of a compiler. It takes the modified source code from language preprocessors that are written in the form of sentences. The lexical analyzer breaks these syntaxes into a series of tokens, by removing any whitespace or comments in the source code.

If the lexical analyzer finds a token invalid, it generates an error. The lexical analyzer works closely with the syntax analyzer. It reads character streams from the source code, checks for legal tokens, and passes the data to the syntax analyzer when it demands.



Token	Sample Lexemes	Informal Description of Pattern
const	const	const
if	if	if
relation	<, <=, =, < >, >, >=	< or <= or = or < > or >= or >
id	pi, count, D2	letter followed by letters and digits
num	3.1416, 0, 6.02E23	any numeric constant
literal	"core dumped"	any characters between " and " except "

Classifies Pattern

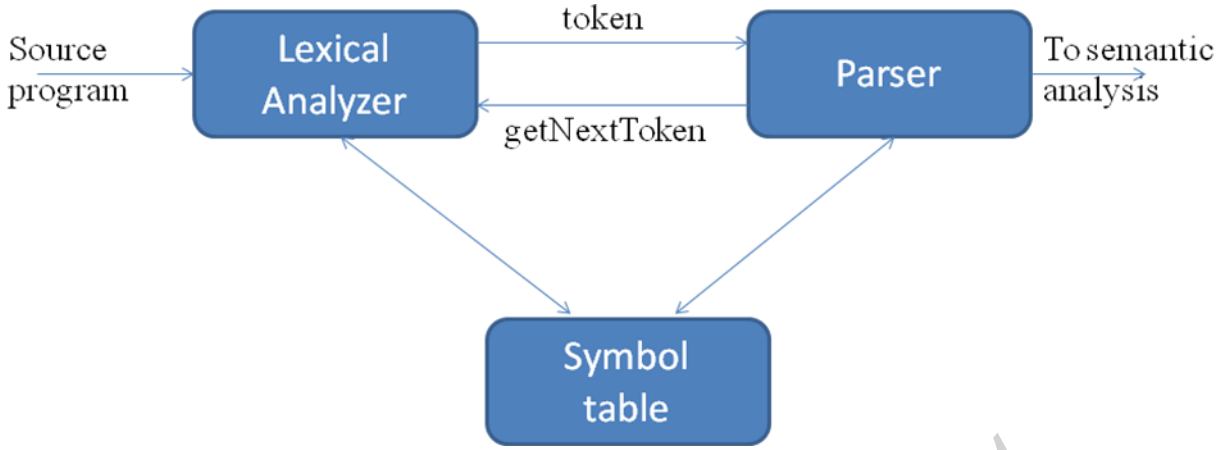
Actual values are critical. Info is :
1. Stored in symbol table
2. Returned to parser

The role of lexical analyzer

- Remove comments and white spaces (aka scanning)
- Macros expansion
- Read input characters from the source program
- Group them into lexemes
- Produce as output a sequence of tokens
- Interact with the symbol table
- Correlate error messages generated by the compiler with the source program

1. Main Task:
   - Its main task is to read the *input characters* and produce as output a sequence of *tokens* that the parser uses for syntax analysis
   - It also performs certain secondary tasks such as *stripping out comments* and *white space* and *correlating error messages* with the source program
2. Interaction of Lexical Analyzer with Parser

Prof(Dr) Anil Kumar



3. Processes in lexical analyzers
    - Scanning
        - Pre-processing
            - Strip out comments and white space
            - Macro functions
    - Correlating error messages from compiler with source program
            - A line number can be associated with an error message
4. Lexical analysis
5. Terms of the lexical analyzer
    - Token
        - Types of words in source program
        - Keywords, operators, identifiers, constants, literal strings, punctuation symbols(such as commas,semicolons)
    - Lexeme
        - Actual words in source program
    - Pattern
            - A rule describing the set of lexemes that can represent a particular token in source program
            - **Relation** {<.<=,>,>=,==,<>}
6. Attributes for Tokens
    - A pointer to the symbol-table entry in which the information about the token is kept
        - E.g E=M*C**2
            - <**id**, pointer to symbol-table entry for E>
                - <**assign_op**,>
            - <**id**, pointer to symbol-table entry for M>
                - <**multi_op**,>
            - <**id**, pointer to symbol-table entry for C>
                - <**exp_op**,>
            - <**num**,integer value 2>
7. Lexical Errors
    - Deleting an extraneous character
    - Inserting a missing character
    - Replacing an incorrect character by a correct character

Prof(Dr) Anil Kumar

- Transposing two adjacent characters(such as , fi=>if)
- Pre-scanning

8. Input Buffering
    - Two-buffer input scheme to look ahead on the input and identify tokens
    - Buffer pairs
    - Sentinels(Guards)

Why to separate Lexical analysis and parsing

1. Simplicity of design
2. Improving compiler efficiency
3. Enhancing compiler portability

There are times when a lexical analyzer needs to look ahead several characters beyond the lexeme for a token before a match can be announced.
Buffering techniques can be used to reduce the overhead required to process input characters.
The buffer is divided into two *N*-character halves.

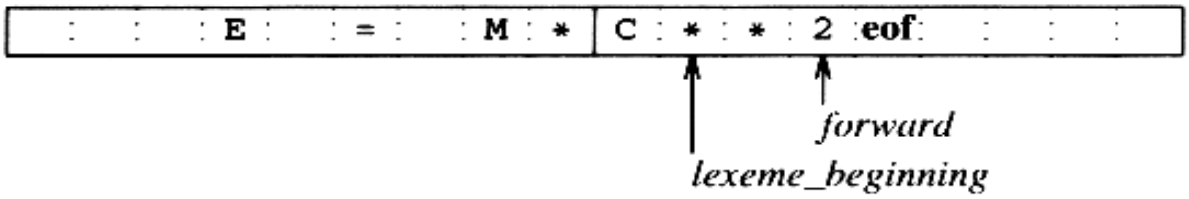


Fig. 3.3. An input buffer in two halves.

- *N* input characters are read into each half of the buffer with one read command.
- If fewer than *N* characters remain in the input then a special character **eof** is read into the buffer.
- Two pointers are maintained. Initially, both pointers point to the *first character* of the next lexeme. The forward pointer scans ahead until a match for a pattern is found. After the lexeme is processed, both pointers are set to the character immediately past the lexeme.
- If the forward pointer is about to move *past the halfway mark*, the right half is filled with *N* new characters.
- If the forward pointer is about to move past the right end of the buffer, the left half is filled with *N* new characters.
- Lookahead is *limited* by the length of the buffer.

```
if forward at end of first half then begin
        reload second half;
        forward := forward + 1
end
else if forward at end of second half then begin
        reload first half;
        move forward to beginning of first half
end
else forward := forward + 1;
```

Fig. 3.4. Code to advance forward pointer.

Sentinels to Improving Input Buffering

- at the ends of buffer halves, we need *two tests* for each advance of the forward pointer. We can reduce it to *one* test if we extend each buffer half to hold the special characters **eof** at the enExcept d of each

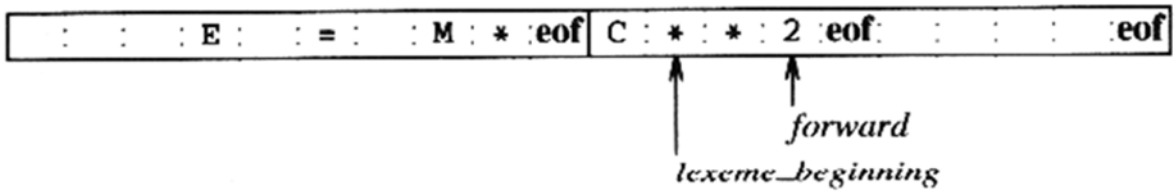


Fig. 3.5. Sentinels at end of each buffer half.

```
        Forward:= forward + 1
If  forward != EOF then begin
        If forward at end of first half then begin
                Reload Second Half
                Forward := Forward + 1
        End
Elseif Forward at end of Second Half then begin
        Reload first Half
        Move Forward to the beginning of first Half
```

```
End
Else
        Terminate Lexical Analysis
End.
```

- Sometimes lexical analyzer needs to look ahead some symbols to decide about the token to return
  - In C language: we need to look after -, = or < to decide what token to return
  - In Fortran: DO 5 I = 1.25
- We need to introduce a two buffer scheme to handle large look-aheads safely
- Because of the amount of time taken to process characters and the large number of characters that must be processed during the compilation of a large source program,
- Specialized buffering techniques have been developed to reduce the amount of overhead required to process a single input character.

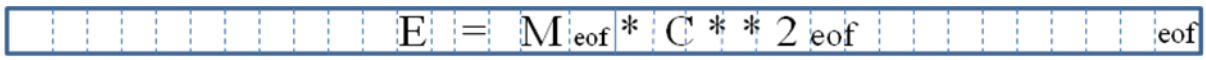
Two pointers to the input are maintained

- Pointer *Lexeme Begin*, marks the beginning of the current lexeme, whose extent we are attempting to determine
- Pointer *Forward*, scans ahead until a pattern match is found.
- Once the next lexeme is determined, *forward* is set to character at its right end.
- Then, after the lexeme is recorded as an attribute value of a token returned to the parser, *Lexeme Begin* is set to the character immediately after the lexeme just found.

Sentinels:

- If we use the scheme of Buffer pairs we must check, each time we advance forward, that we have not moved off one of the buffers; if we do, then we must also reload the other buffer.
- Thus, for each character read, we make two tests: one for the end of the buffer, and one to determine what character is read (the latter may be a multiway branch).
- We can combine the buffer-end test with the test for the current character if we extend each buffer to hold a sentinel character at the end.
- The sentinel is a special character that cannot be part of the source program, and a natural choice is the character EOF.

Prof (Dr) Anil Kumar




```
Switch (*forward++) {
    case eof:
            if (forward is at end of first buffer) {
                    reload second buffer;
                    forward = beginning of second buffer;
            }
            else if {forward is at end of second buffer) {
                    reload first buffer;\
                    forward = beginning of first buffer;
            }
            else /* eof within a buffer marks the end of input */
                    terminate lexical analysis;
            break;
    cases for the other characters;
}
```

Specification of Token

- We need a formal way to specify patterns: regular expressions
- Alphabet: any finite set of symbols
- String over alphabet: finite sequence of symbols drawn from that alphabet
- Language: countable set of strings over some fixed alphabet
- Regular expressions are an important notation for specifying *patterns*.
- While they cannot express all possible patterns, they are very effective in specifying those type of patterns that we actually need for tokens.
- Regular expressions are an important notation for specifying *patterns*.
- In theory of compilation regular expressions are used to formalize the specification of tokens
- Regular expressions are means for specifying regular languages
- Example:
    - Letter_(letter_ | digit)*
- Each regular expression is a pattern specifying the form of strings

1. **Regular Definition of Tokens**
    – Defined in regular expression

    e.g. Id → letter(letter|digit)

    letter →A|B|…|Z|a|b|…|z

    digit →0|1|2|…|9

Notes: Regular expressions are an important notation for specifying patterns. Each pattern matches a set of strings, so regular expressions will serve as as names for sets of strings.

Recognition of tokens

- Starting point is the language grammar to understand the tokens:

stmt -> if expr then stmt
| if expr then stmt else stmt
| ε
expr -> term relop term

Prof (Dr) Anil Kumar

| term
term -> id
| number

- The next step is to formalize the patterns:

digit -> [0-9]
Digits -> digit+
number -> digit(.digits)? (E[+-]? Digit)?
letter -> [A-Za-z_]
id -> letter (letter|digit)*
If -> if
Then -> then
Else -> else
Relop -> < | > | <= | >= | = | <>

- We also need to handle whitespaces: *ws* -> (blank | tab | newline)+

Token Recognition

How can we use concepts developed so far to assist in recognizing tokens of a source language ?

Assume Following Tokens:

if, then, else, relop, id, num

What language construct are they used for ?

Given Tokens, What are Patterns ?

if → if
then → then
else → else
relop → < | <= | > | >= | = | <>
id → letter (letter | digit)*
num → digit + (. digit +) ? (E(+ | -) ? digit +) ?

What does this represent ? What is Σ ?

Grammar:
stmt → |if *expr* then *stmt*
|if *expr* then *stmt* else *stmt*
|∈
expr → *term* relop *term* | *term*
term → id | num

Overall

Prof(Dr) Anil Kumar

Regular Expression	Token	Attribute-Value
ws	-	-
if	if	-
then	then	-
else	else	-
id	id	pointer to table entry
num	num	pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
<>	relop	NE
>	relop	GT
>=	relop	GE

Note: Each token has a unique token identifier to define category of lexemes

Tokens Recognition

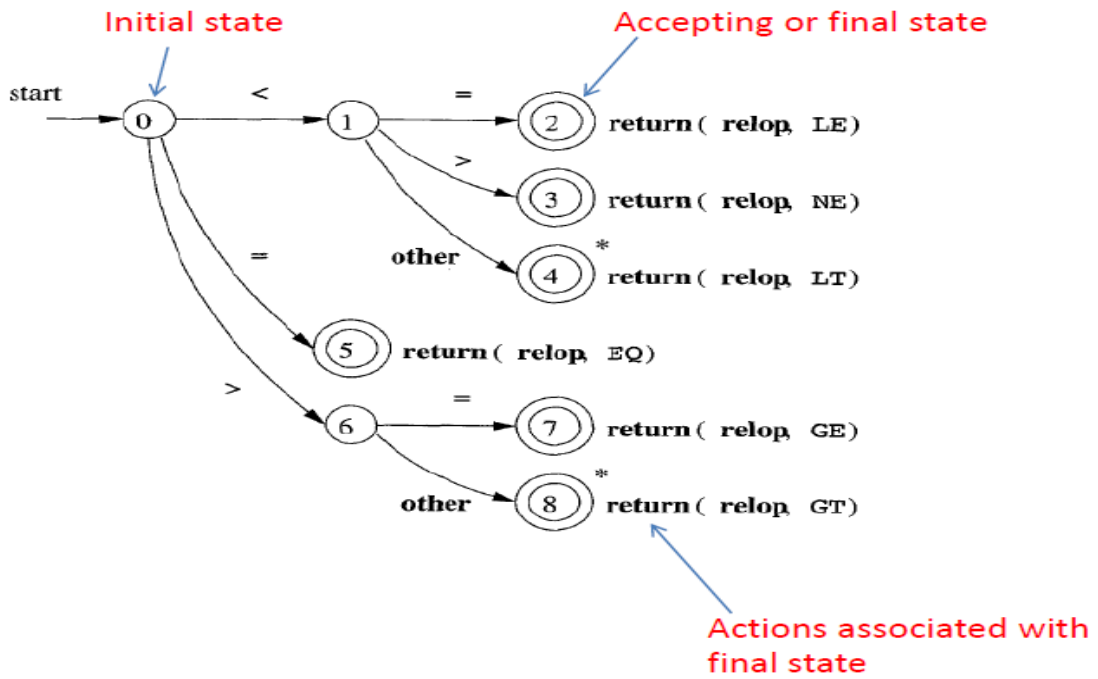
$$
\begin{array}{lcl}
stmt & \rightarrow & \textbf{if}\ expr\ \textbf{then}\ stmt \\
 & | & \textbf{if}\ expr\ \textbf{then}\ stmt\ \textbf{else}\ stmt \\
 & | & \epsilon \\
expr & \rightarrow & term\ \textbf{relop}\ term \\
 & | & term \\
term & \rightarrow & \textbf{id} \\
 & | & \textbf{number}
\end{array}
$$

$$
\begin{array}{rcl}
digit & \rightarrow & [0\text{-}9] \\
digits & \rightarrow & digit^{+} \\
number & \rightarrow & digits\ (\ .\ digits)?\ (\ \mathtt{E}\ [+-]?\ digits\ )? \\
letter & \rightarrow & [\mathtt{A\text{-}Za\text{-}z}] \\
id & \rightarrow & letter\ (\ letter \mid digit\ )^{*} \\
if & \rightarrow & \mathtt{if} \\
then & \rightarrow & \mathtt{then} \\
else & \rightarrow & \mathtt{else} \\
relop & \rightarrow & \mathtt{<} \mid \mathtt{>} \mid \mathtt{<=} \mid \mathtt{>=} \mid \mathtt{=} \mid \mathtt{<>} \\
ws & \rightarrow & (\ \textbf{blank} \mid \textbf{tab} \mid \textbf{newline}\ )^{+}
\end{array}
$$

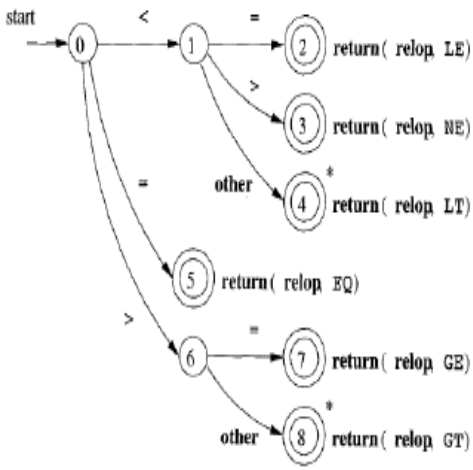
LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any *ws*	–	–
if	**if**	–
then	**then**	–
else	**else**	–
Any *id*	**id**	Pointer to table entry
Any *number*	**number**	Pointer to table entry
<	**relop**	LT
<=	**relop**	LE
=	**relop**	EQ
<>	**relop**	NE
>	**relop**	GT
>=	**relop**	GE

Implementation of transition diagram

- Intermediate step in constructing lexical analyzer
- Convert patterns into flowcharts called transition diagrams
- – nodes or circles: called states
- – Edges: directed from state to another, labeled by symbols




```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```



Finite automata

Finite automata is a state machine that takes a string of symbols as input and changes its state accordingly. Finite automata is a recognizer for regular expressions. When a regular expression string is fed into finite automata, it changes its state for each literal.

Finite Automata(FA) is the simplest machine to recognize patterns.It is used to characterize a Regular Language, for example: /baa+!/.
Also it is used to analyze and recognize Natural language Expressions. The finite automata or finite state machine is an abstract machine that has five elements or tuples. It has a set of states and rules for moving from one state to another but it depends upon the applied input symbol. Based on the states and the set of rules the input string can be either accepted or rejected. Basically, it is an abstract model of a digital computer which reads an input string and changes its internal state depending on the current input symbol. Every automaton defines a language i.e. set of strings it accepts. The following figure shows some essential features of general automation.

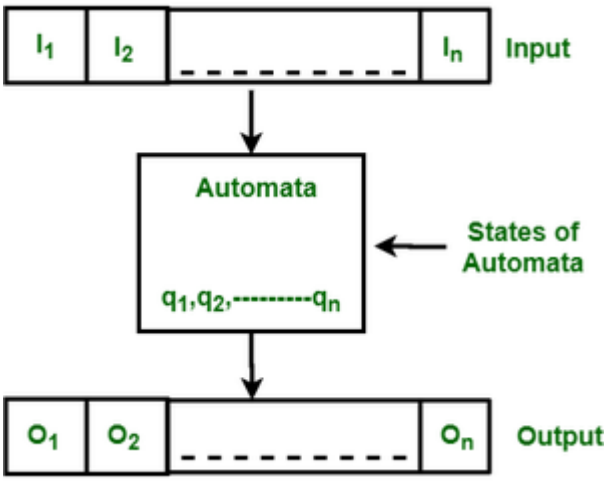


Figure: *Features of Finite Automata*

The above figure shows the following features of automata:

1. Input
2. Output
3. States of automata
4. State relation
5. Output relation

A language for specifying Lexical Analyzers

LEX is one such lexical analyzer generator which produces C code based on the token specifications. This tool has been widely used to specify lexical analyzers for a variety of languages. We refer to the tool as Lex Compiler, and to its input specification as the Lex language.
There is a wide range of tools for constructing lexical analyzers.

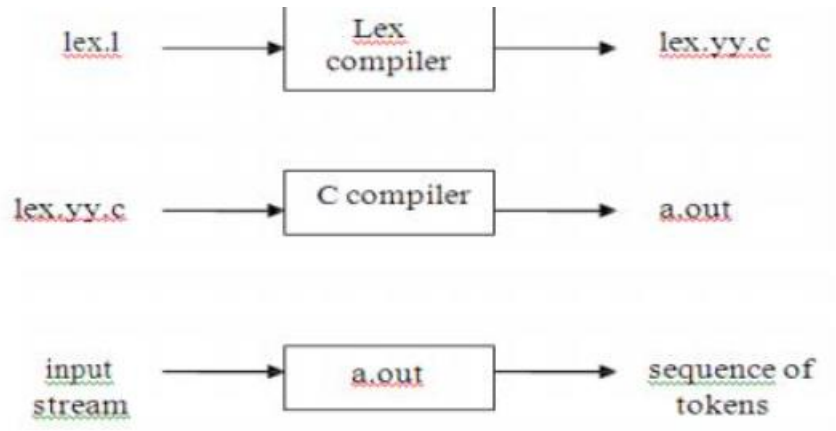
Lex
YACC

Lex is a computer program that generates lexical analyzers. Lex is commonly used with the yacc parser generator.

Creating a lexical analyzer

- First, a specification of a lexical analyzer is prepared by creating a program lex.l in the Lex language. Then, lex.l is run through the Lex compiler to produce a C program lex.yy.c.

- Finally, lex.yy.c is run through the C compiler to produce an object progra m a.out, which is the lexical analyzer that transforms an input stream into a sequence of tokens.



Creating a lexical analyzer with lex

Fig1.11 Creating a lexical analyzer with lex

Lex Specification

A Lex program consists of three parts:

{ definitions }
%%
{ rules }
%%
{ user subroutines }

Definitions include declarations of variables, constants, and regular definitions

Ø **Rules** are statements of the form

p1 {action1}
p2 {action2}
…
pn {actionn}

where pi is regular expression and actioni describes what action the lexical analyzer should take when pattern pi matches a lexeme. Actions are written in C code.

□

□□**User subroutines**□are auxiliary procedures needed by the actions. These can be compiled□separately and loaded with the lexical analyzer.

YACC- YET ANOTHER COMPILER-COMPILER

Yacc provides a general tool for describing the input to a computer program. The Yacc user specifies the structures of his input, together with code to be invoked as each such structure is recognized.

Yacc turns such a specification into a subroutine that handles the input process; frequently, it is convenient and appropriate to have most of the flow of control in the user's application handled by this subroutine.

Design and implementation of lexical analyzer

Lexical Analysis is the very first phase in the compiler designing. A Lexer takes the modified source code which is written in the form of sentences . In other words, it helps you to convert a sequence of characters into a sequence of tokens. The lexical analyzer breaks this syntax into a series of tokens. It removes any extra space or comment written in the source code.
Programs that perform Lexical Analysis in compiler design are called lexical analyzers or lexers. A lexer contains tokenizer or scanner. If the lexical analyzer detects that the token is invalid, it generates an error. The role of Lexical Analyzer in compiler design is to read character streams from the source code, check for legal tokens, and pass the data to the syntax analyzer when it demands.

Prof(Dr) Anil Ku